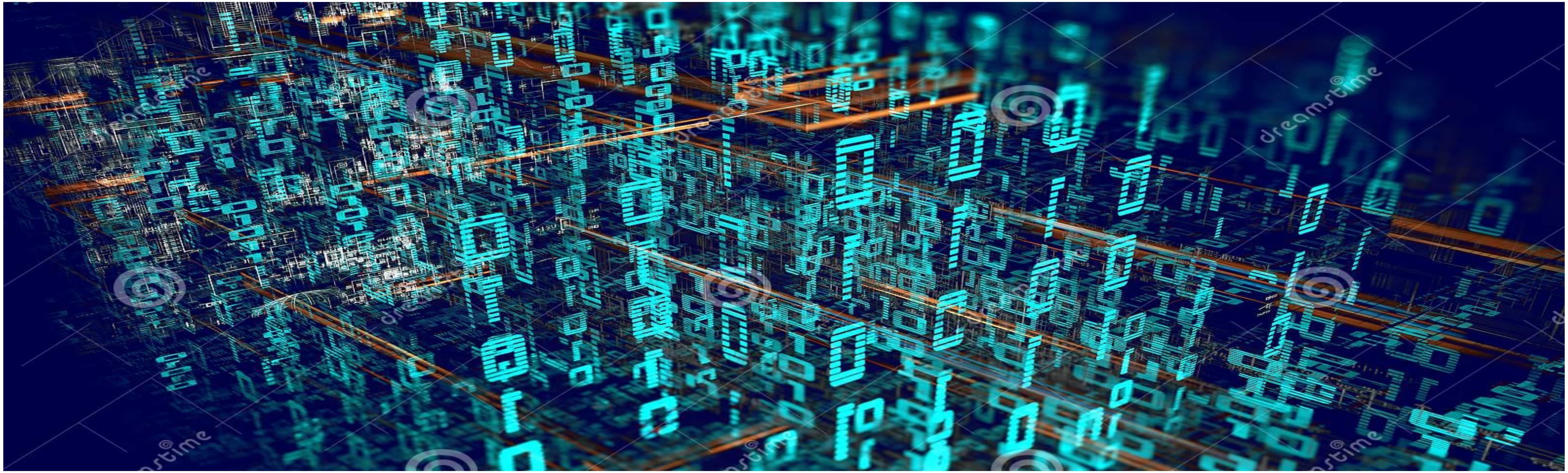
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE307L_COMPILER DESIGN



CODE OPTIMIZATION

Dr. B.V. Baiju,
SCOPE,Assistant Professor
VIT, Vellore

Loops in Flow Graphs

- A graph representation of three-address statements, called a flow graph, is useful for understanding code-generation algorithms, even if the graph is not explicitly constructed by a code-generation algorithm.
- **Nodes** in the flow graph represent ***computations***, and the **edges** represent the ***flow of control.***

1. Dominators

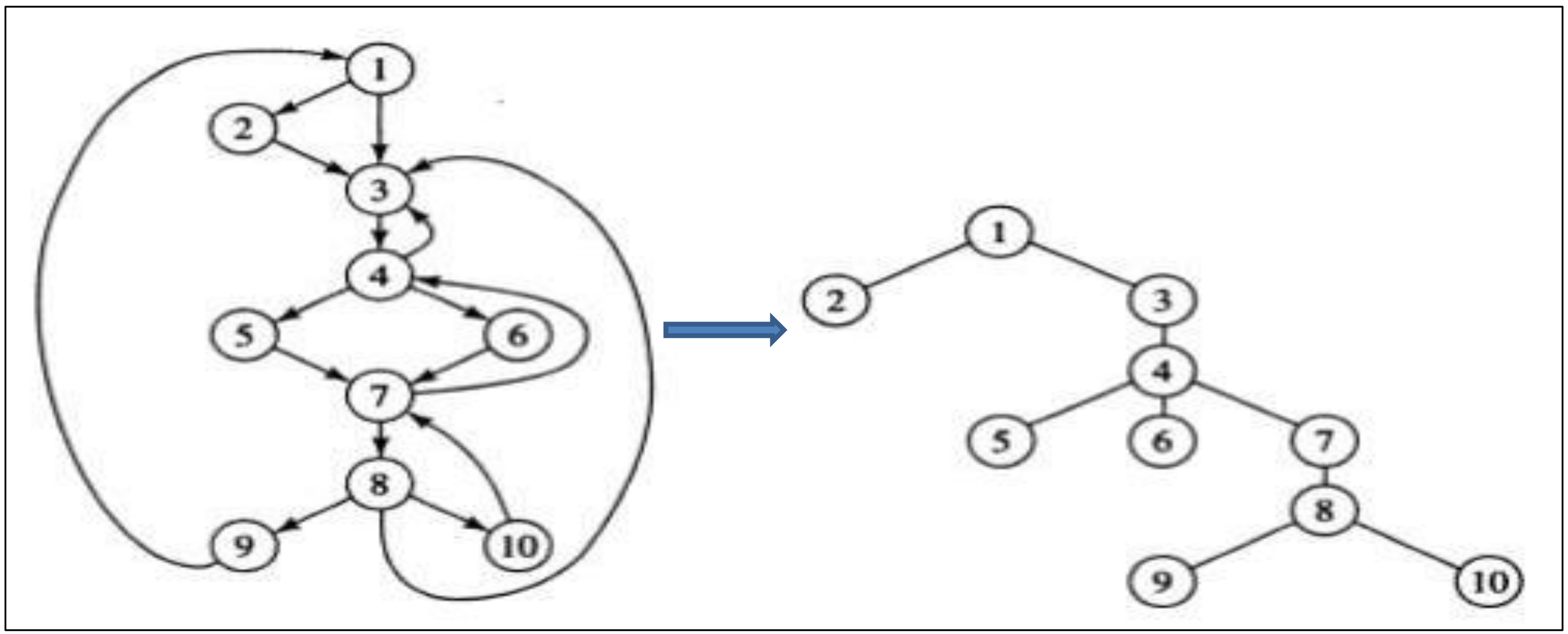
- Dominators are used to ***identify the leaders*** of a control flow graph and to optimize loops.
- Given two nodes, 'd' and 'n', we say, node 'd' dominates 'n', denotes as

"d dom n",

if every path from the initial node of the control flow graph (CFG) to 'n' goes through 'd'.

- The loop entry dominates all nodes in the loop.

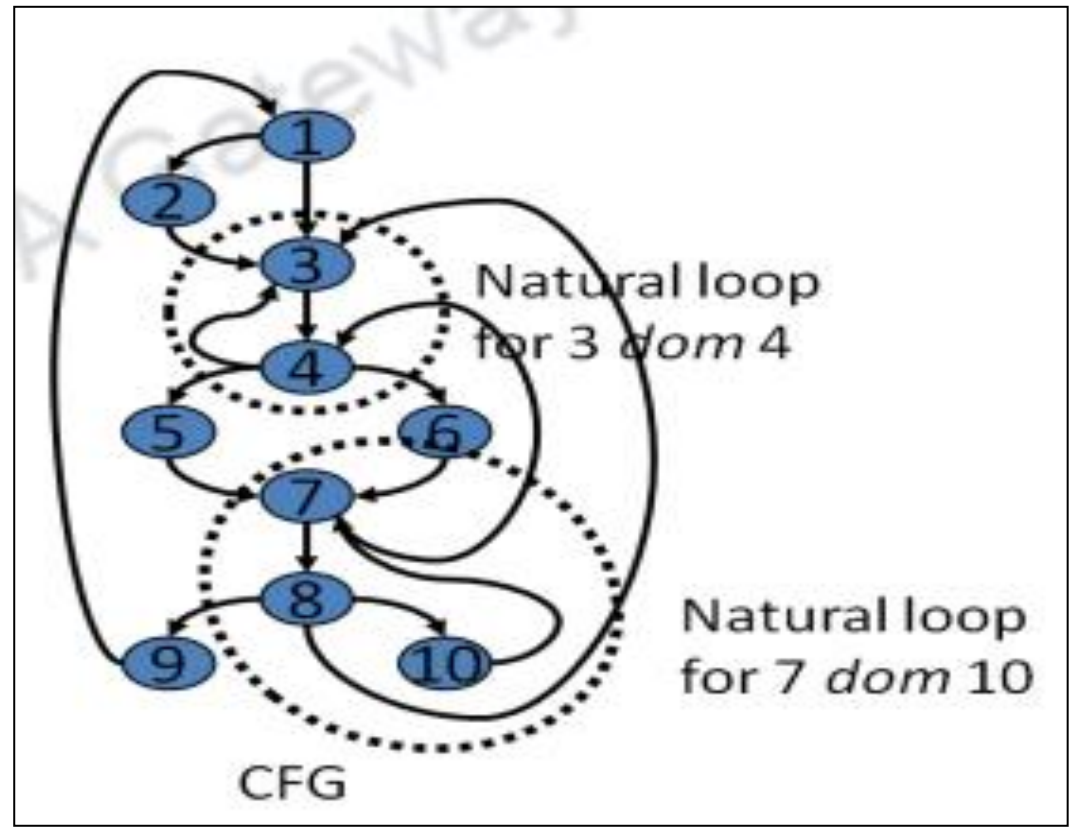
- The parent of each other node is its **immediate dominator**.
- The immediate dominator ‘m’ of a node ‘n’ is the last dominator on the path from the initial node to ‘n.
- If d ≠ n and “d dom n” then “d dom m”.
- The way of presenting dominator information is in a tree, called the **dominator tree**, in which
  - ***The initial node is the root.***
  - ***The parent of each other node is its immediate dominator.***
  - ***Each node d dominates only its descendants in the tree.***



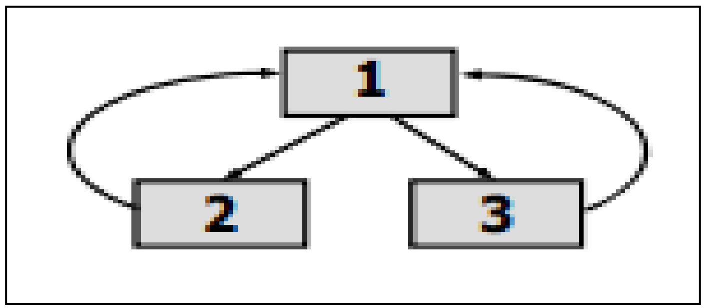
D(1)={1}
D(2)={1,2}
D(3)={1,3}
D(4)={1,3,4}
D(5)={1,3,4,5}
D(6)={1,3,4,6}
D(7)={1,3,4,7}
D(8)={1,3,4,7,8}
D(9)={1,3,4,7,8,9}
D(10)={1,3,4,7,8,10}

2. Natural Loops

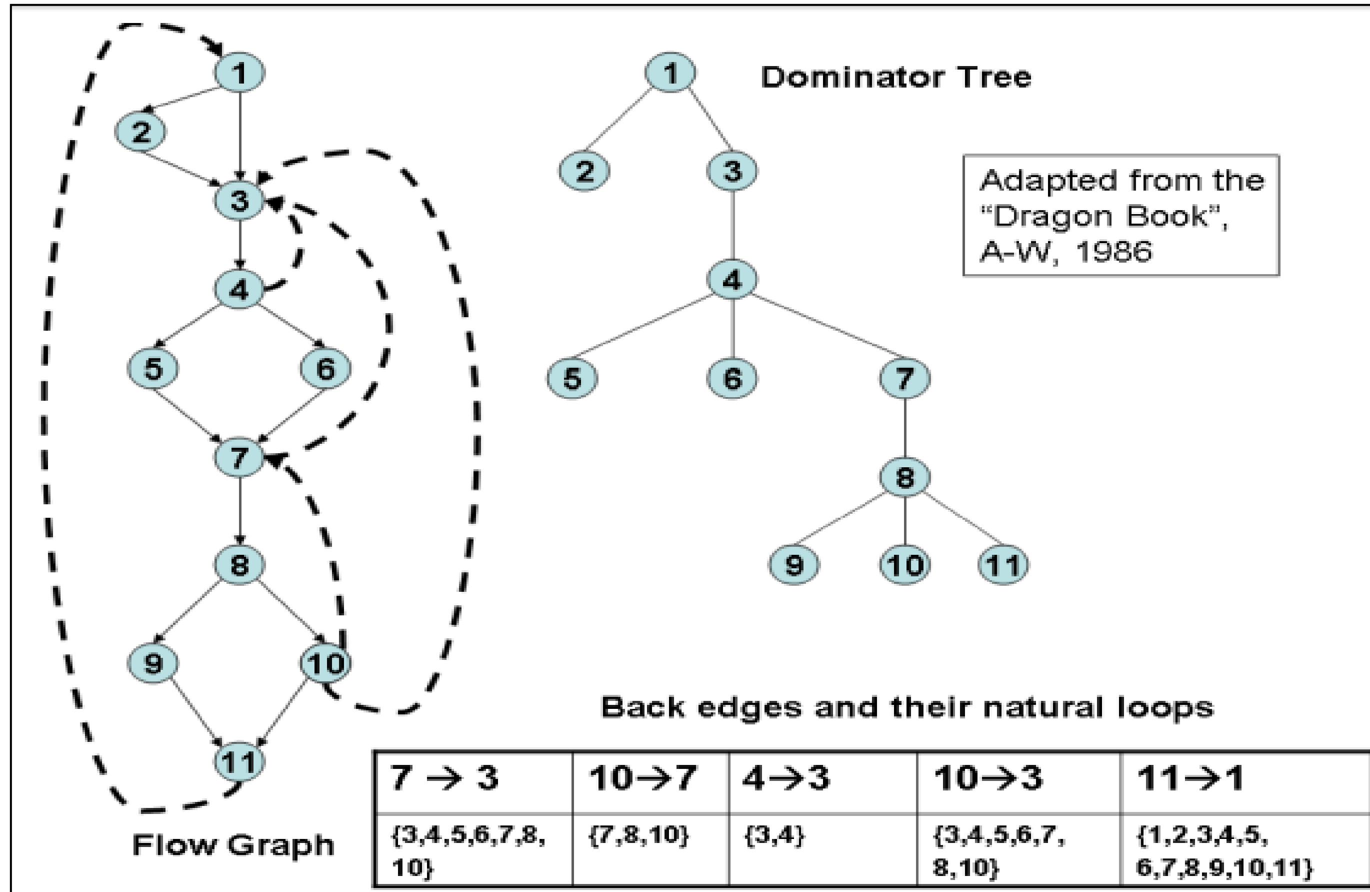
- One application of dominator information is in determining the loops of a flow graph suitable for improvement.
- There are two essential properties of loops:
  - A ***loop must have a single entry point***, called the **header**. This entry point-dominates all nodes in the loop, or it would not be the sole entry to the loop.
  - There must be ***at least one way to iterate the loop***(i.e.)at least one path back to the header.
    - One way to find all the loops in a flow graph is to search for edges in the flow graph whose heads dominate their tails.
    - If **a→b** is an edge, ***b is the head*** and ***a is the tail***.
    - These types of edges are called as **back edges**.



- For any two natural loops in the flow graph, one of the following is true:
    1. ***They are disjoint***
    2. ***They are nested***
    3. ***They have the same header***
- If two loops have the same header and none is nested in the other, combine all nodes into a single loop



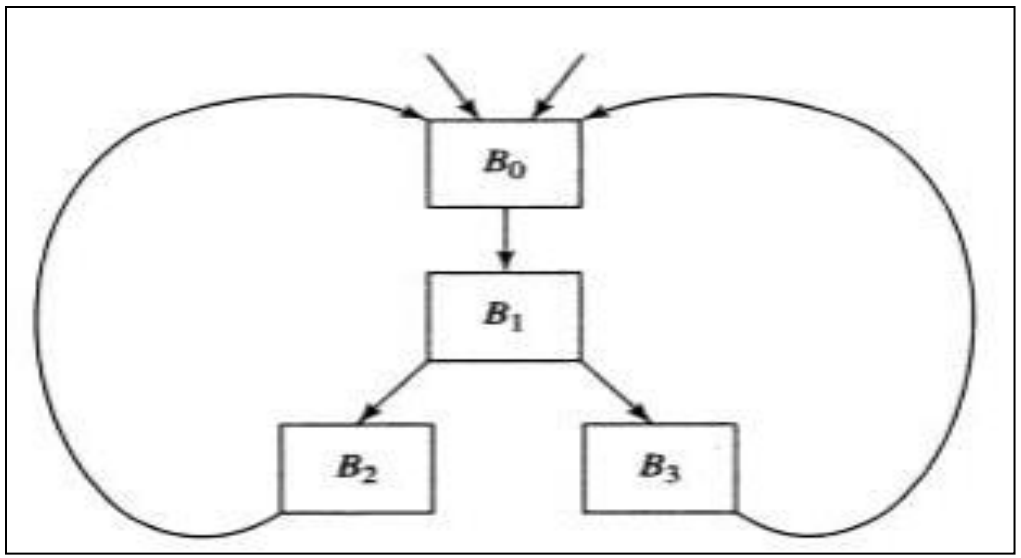
Two loops: {1,2} and {1,3}
Combine into one loop: {1,2,3}

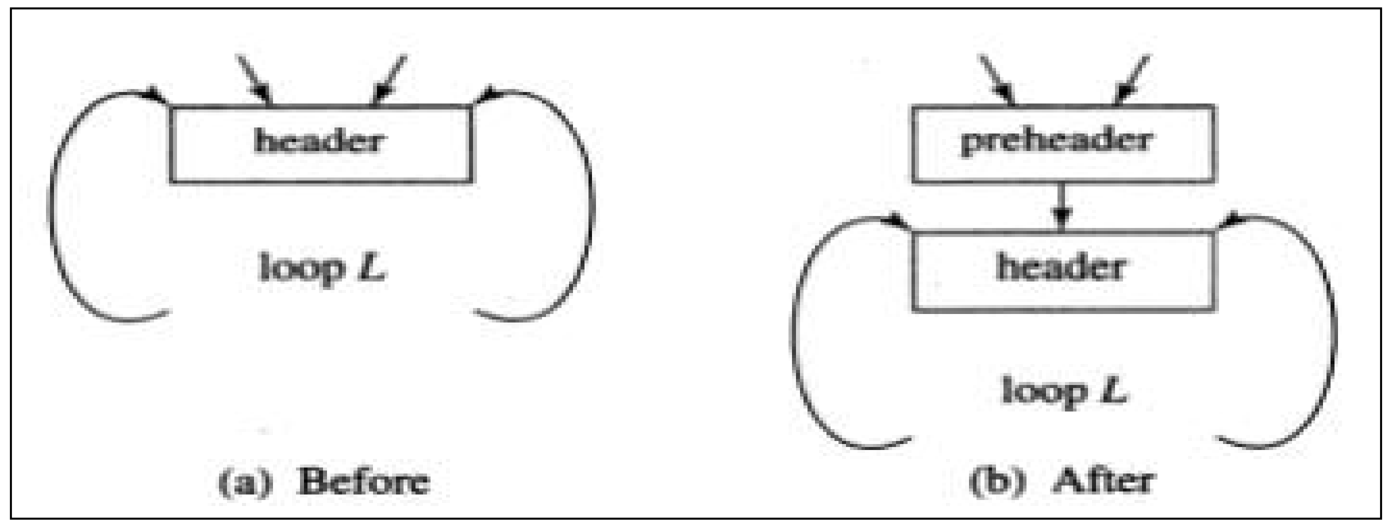


7 → 3	10→7	4→3	10→3	11→1
{3,4,5,6,7,8, 10}	{7,8,10}	{3,4}	{3,4,5,6,7, 8,10}	{1,2,3,4,5, 6,7,8,9,10,11}

3. Pre-Headers

- Identifying natural loops and dominators help in loop optimization.
- The pre-header has only the ***header as successor***, and all edges which formerly entered the header of L from outside L instead enter the pre-header.
- Edges from inside loop L to the header are not changed.
- Initially the pre-header is empty, but transformations on L may place statements in it.

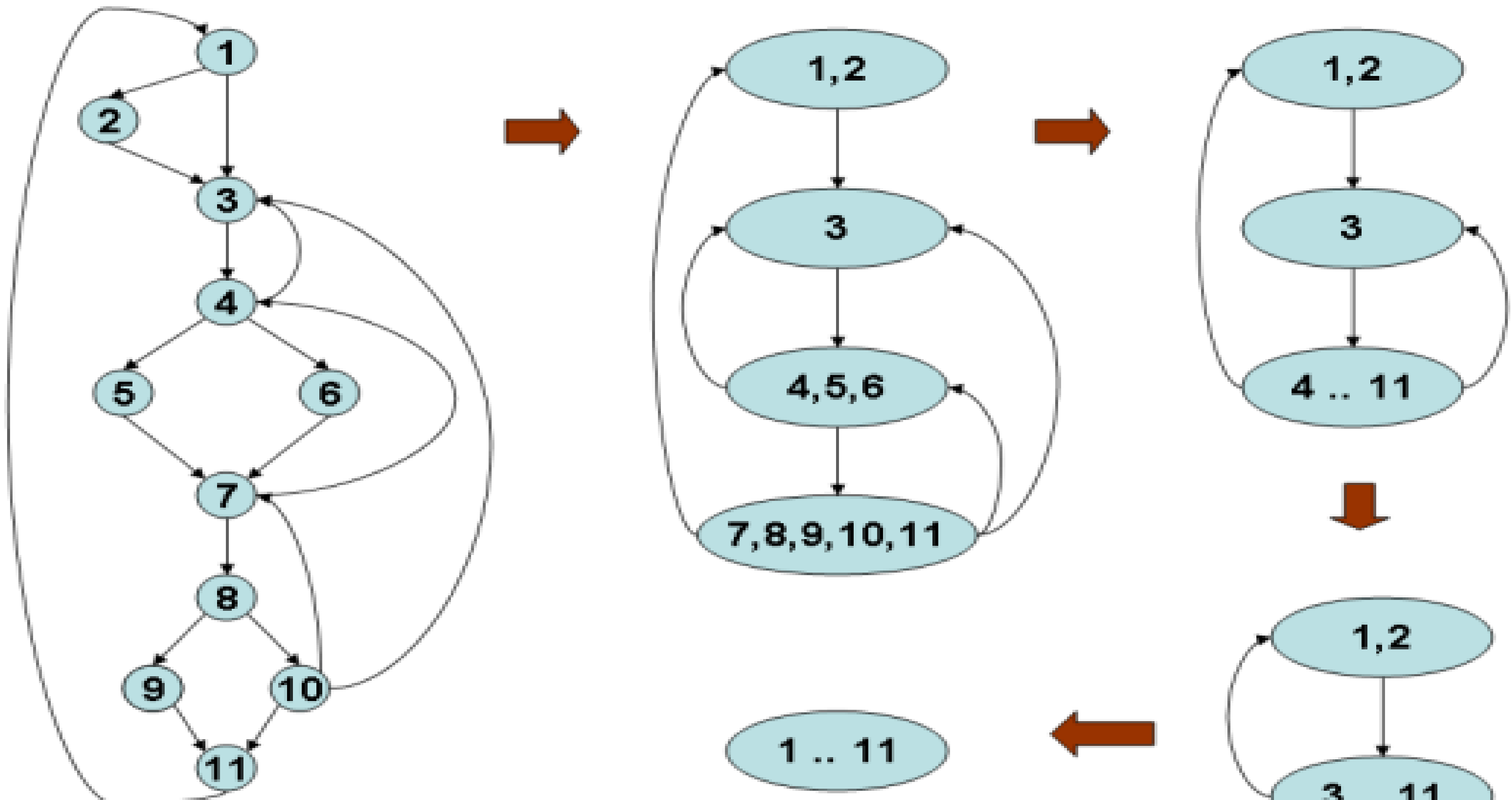




Two loops with the same header

4. Reducible flow graphs

- Reducible flow graphs are special flow graphs, for which
  - ***several code optimization transformations are especially easy to perform***
  - ***loops are unambiguously defined***
  - ***dominators can be easily calculated***
  - ***data flow analysis problems can also be solved efficiently.***
- Exclusive use of structured flow-of-control statements such as if-then-else, while-do, continue, and break statements produces programs whose flow graphs are always reducible.
- A control flow graph is said to be reducible, if the forward edges form an acyclic graph where every node can be reached from the initial node G.
- The most important properties of reducible flow graphs are that
  1. **There are no jumps into the middle of loops from outside**
  2. **The only entry to a loop is through its header**



Flow Graph

I(1) = {1,2}; I(3) = {3}
I(4) = {4,5,6}; I(7) = {7,8,9,10,11}

Adapted from
"The Dragon Book", A-W 1986